



# *Algorithm Project*

## Triangle Project

### Team members:

20240493

- صالح أسامة صالح

20240047

- احمد طاهر عبد العظيم

20240078

- احمد محمد سمير

20240474

- شادي هاني عبد المنعم

20240979

- مصطفى وليد سيد

20240577

- عبد الله عماد كارم

## **First Algorithm (Non-Recursive):**

### **Idea:**

The algorithm sorts the array using Bubble Sort, then checks every 3 consecutive numbers.

If the sum of the first two numbers is greater than the third number, then a triangle exists.

### **Pseudo-code:**

BubbleSort(A)

1. For i from 0 to length(A)-1
2. For j from 0 to length(A)-i-2
3. If  $A[j] > A[j+1]$
4. Swap  $A[j]$  and  $A[j+1]$

TriangleNonRecursive(A)

1. BubbleSort(A)
2. For i from 0 to length(A)-3
3. If  $A[i] + A[i+1] > A[i+2]$
4. Return 1
5. Return 0

Display on Screen

1. Display Enter Inputs
2. For Loop to read inputs
3. Call (triangleNonRecursive)
4. Display output

## Implementation:

```
#include <stdio.h>

/* Bubble Sort */

void sortArray(int A[], int N)
{
    int i, j, temp;
    for(i = 0; i < N - 1; i++)
    {
        for(j = 0; j < N - i - 1; j++)
        {
            if(A[j] > A[j + 1])
            {
                temp = A[j];
                A[j] = A[j + 1];
                A[j + 1] = temp;
            }
        }
    }
}

/* Triangle Check */

int triangleNonRecursive(int A[], int N)
{
    int i;

    /* Sort the array */
    sortArray(A, N);

    /* Check triangle condition */
    for(i = 0; i < N - 2; i++)
```

```

{
    if(A[i] + A[i + 1] > A[i + 2])
    {
        return 1;
    }
}
return 0;
}

int main()
{
    int N;
    int i;
    printf("Non-Recursive Algorithm to Check is there a Triangle \n");
    printf("\nEnter number of Inputs: ");
    scanf("%d", &N);
    int A[N];
    /* Input array elements */
    for(i = 0; i < N; i++)
    {
        printf("Enter side %d: ", i + 1);
        scanf("%d", &A[i]);
    }
    int result;
    result = triangleNonRecursive(A, N);
    if(result == 1)
    {
        printf("\n {1} Triangle exists\n");
    }
}

```

```
}  
else  
{  
    printf("\n {0} Triangle does not exist\n");  
}  
return 0;  
}
```

**Analysis & complexity (steps):**

Sorting Complexity:  $O(n^2)$

Traversal Complexity:  $O(n)$

Final Time Complexity:  $O(n^2)$

Space Complexity:  $O(1)$

## **Second Algorithm (Recursive):**

### **Idea:**

The algorithm sorts the array, then uses recursion to check every triplet.

### **Pseudo-code:**

MergeSort(A)

1. If  $\text{length}(A) \leq 1$
2. Return A
3.  $\text{mid} = \text{length}(A) / 2$
4.  $\text{left} = \text{MergeSort}(\text{first half})$
5.  $\text{right} = \text{MergeSort}(\text{second half})$
6. Return Merge(left, right)

CheckTriangle(A, i)

1. If  $i \geq \text{length}(A) - 2$
2. Return 0
3. If  $A[i] + A[i+1] > A[i+2]$
4. Return 1
5. Return CheckTriangle(A, i+1)

TriangleRecursive(A)

1.  $A = \text{MergeSort}(A)$
2. Return CheckTriangle(A, 0)

Display on Screen

1. Display Enter Inputs
2. For Loop to read inputs
3. Call (triangleRecursive)
4. Display output

### Implementation:

```
#include <stdio.h>
```

```
void bubbleSortArray(int A[], int N)
```

```
{  
    int i, j, temp;  
  
    for(i = 0; i < N - 1; i++)  
    {  
        for(j = 0; j < N - i - 1; j++)  
        {  
            if(A[j] > A[j + 1])  
            {  
                temp = A[j];  
                A[j] = A[j + 1];  
                A[j + 1] = temp;  
            }  
        }  
    }  
}
```

```
/* Recursive Function */
```

```
int checkTriangle(int A[], int N, int i)
```

```
{  
  
    if(i >= N - 2)
```

```

{
    return 0;
}

/* Triangle condition */
if(A[i] + A[i + 1] > A[i + 2])
{
    return 1;
}

return checkTriangle(A, N, i + 1);
}

int triangleRecursive(int A[], int N)
{

    bubbleSortArray(A, N);

    return checkTriangle(A, N, 0);
}

int main_0()
{
    int N;

```

```

int i;

printf("Recursive Algorithm to Check is there a Triangle \n");
printf("\nEnter number of Inputs : ");
scanf("%d", &N);

int A[N];

for(i = 0; i < N; i++)
{
    printf("Enter side %d: ", i + 1);
    scanf("%d", &A[i]);
}
int result;
result = triangleRecursive(A, N);
if(result == 1)
{
    printf("\n (1) Triangle exists\n");
}
else
{
    printf("\n (0) Triangle does not exist\n");
}

return 0;
}

```

### Analysis & Complexity:

Sorting Complexity:  $O(n^2)$

Recursive Calls Complexity:  $O(n)$

Final Time Complexity:  $O(n^2)$

Space Complexity:  $O(n)$

### Comparison Between the 2 algorithms:

| Feature           | Non-Recursive                                                  | Recursive                                                                  |
|-------------------|----------------------------------------------------------------|----------------------------------------------------------------------------|
| Technique Used    | Uses loops (for loop) to iterate through the array.            | Uses recursive function calls to repeat the process.                       |
| Working Method    | The algorithm checks each triplet directly using iteration.    | The algorithm checks one triplet, then calls itself to check the next one. |
| Time Complexity   | $O(n^2)$ because Bubble Sort is used.                          | $O(n^2)$ because Bubble Sort is used.                                      |
| Space Complexity  | $O(1)$ because it uses only a few variables..                  | $O(n)$ because recursive calls are stored in the call stack.               |
| Memory Usage      | Very low memory usage.                                         | Higher memory usage because every recursive call consumes stack memory.    |
| Speed in Practice | Faster in real execution                                       | Slightly slower because of recursive call overhead.                        |
| Best Use Case     | Better for real applications and performance-critical systems. | Better for learning recursion concepts and recursive thinking.             |